

Event-Driven Trading Bot для MOEX

Ссылка: <https://github.com/vXofi/tradingbot>

Кратко

Событийная торговая система для российского фондового рынка (MOEX): поток русскоязычных новостей преобразуется в торговые решения с подтверждением по микроструктуре рынка и исполнением в песочнице Tinkoff Invest API. Входы --- RSS (6 лент), опционально Telegram, ручной ввод через CLI/дашборд; выходы --- ордера в sandbox, закрытые сделки в SQLite, структурированные JSON-логи и терминальный дашборд. Архитектура event-driven: каждое новостное событие асинхронно проходит цепочку NLP → арбитраж → валидация order flow → исполнение → мониторинг выхода.

Стек

Python 3.10+, asyncio; Tinkoff Invest SDK (gRPC-стримы стакана и сделок); aiohttp+feedparser (RSS); Google Gemini API --- LLM-fallback для NLP; Telethon (опционально); терминальный TUI на rich+plotext; SQLite; pandas/numpy; pytest+pytest-asyncio.

Архитектура

RSS/CLI → NewsParser → SignalArbiter → FlowAnalyzer → OrderManager → PositionTracker → ExitSignal

Ключевые компоненты

- **NewsParser** --- гибридный NLP: keyword-матчинг (64/59 триггеров), корпоративные и макро-события с маршрутизацией через 9 секторов, entity resolution (24 тикера, 98 синонимов/брендов), Gemini как fallback.
- **SignalArbiter** --- дедупликация в окне 60 с и детекция противоречий с открытыми позициями (в т.ч. триггер разворота).
- **FlowAnalyzer** --- подтверждение сигнала по дисбалансу стакана, всплеску объёма и направлению цены; отдельный путь validate_reversal.
- **RiskManager** --- ATR(14)-based SL/TP, sizing под лимит позиции, trailing stop, momentum exit.
- **OrderManager** --- исполнение marketable limit-ордеров в sandbox Tinkoff API.
- **PositionTracker** --- state machine ACTIVE → REVERSING → CLOSED, единый ExitSignal для всех условий выхода.
- **Устойчивость** --- stream_with_reconnect, async_retry, rate limiters (180 req/min Tinkoff, 55 req/min Gemini).
- **Persistence & observability** --- SQLite trades.db, EventLogger (JSON + ротация), 3-страничный Rich-дашборд (сигналы/графики/история).

Проверка и качество

- 430 pytest-сценариев (оркестратор, E2E pipeline, NLP eval); CI --- GitHub Actions.
- **NLP** (38 заголовков, rules-only): accuracy 100 %, macro-F1 1,0; ticker recall 100 %; FPR 0 % на negative-set. Corpus hand-labeled, не out-of-sample RSS.
- **Backtest** (20 сигналов, исторические свечи): win rate 40 %, profit factor 2,97, max drawdown 1,68 sim units --- **не** real PnL, order-flow не моделируется.
- Live-режим --- sandbox по умолчанию (USE_SANDBOX=true).

Инженерная ценность

Проект демонстрирует end-to-end инженерии ИИ-системы: гибридный NLP (rules + LLM), интеграция с live market data, event-driven оркестрация, управление риском и устойчивость к сбоям внешних API. Для инженерного трека важны не заявления о доходности, а воспроизводимая архитектура, тесты, observability и честное разделение sandbox/backtest vs production.

Ограничения

- Реальная торговля и production PnL **не подтверждались**; стратегия не запускалась на боевом контуре.
- Результаты backtest --- симуляция на исторических свечах по синтетическому CSV; без комиссий, проскальзывания и order-flow фильтрации; не доказывают доходность.
- NLP eval corpus --- hand-labeled, rules-only; LLM-fallback и live order-flow не входят в эти метрики.
- Sandbox-исполнение не эквивалентно реальной ликвидности и задержкам биржи.

Запуск: `python main.py · python main.py --backtest data/backtest_signals_sample.csv · pytest tests/`
-v